

Python OOP learning pathway

Phase 1 — Python object fundamentals

1. Classes and objects
2. class execution in Python
3. `__init__`
4. `self`
5. Instance attributes
6. Class attributes
7. Instance namespace
8. `__dict__`
9. Dynamic attribute creation
10. Attribute lookup mechanism
11. `getattr()`, `setattr()`, `hasattr()`, `delattr()`
12. Everything is an object

Hands-on: build Student, Employee, and BankAccount models and inspect their runtime state.

Phase 2 — Methods in extreme depth

1. Instance methods
2. Class methods
3. Static methods
4. `@classmethod`
5. `@staticmethod`
6. Method binding
7. Bound methods
8. Unbound function concept
9. How Python automatically passes `self`
10. Functions vs methods internally
11. `__func__`

12. __self__

This is very important because Python method dispatch is quite different from Java.

Hands-on: manually inspect bound methods and recreate method invocation behavior.

Phase 3 — Encapsulation and attribute control

1. Public attributes
2. _protected convention
3. __private
4. Name mangling
5. Why Python has no true private
6. Getter/setter approach
7. property()
8. @property
9. Property setters
10. Property deleters
11. Computed attributes
12. Attribute validation

Hands-on: build a properly encapsulated BankAccount.

Phase 4 — Inheritance

1. Basic inheritance
2. Method overriding
3. super()
4. Constructor inheritance
5. Constructor overriding
6. Parent constructor calls
7. Multilevel inheritance
8. Hierarchical inheritance
9. Multiple inheritance
10. Diamond problem

11. MRO
12. C3 linearization
13. Class.__mro__
14. mro()
15. Cooperative inheritance
16. Why super() does not simply mean "parent"

This phase will go much deeper than normal Python courses.

Hands-on: manually predict MRO and build multiple-inheritance hierarchies.

Phase 5 — Polymorphism

1. Method overriding
2. Duck typing
3. Runtime polymorphism
4. Operator polymorphism
5. Protocol-style programming
6. hasattr based polymorphism
7. EAFP vs LBYL
8. Polymorphism without inheritance

Hands-on: payment processing system using duck typing.

Phase 6 — Abstraction

1. Abstract classes
2. ABC
3. ABCMeta
4. @abstractmethod
5. Abstract properties
6. Abstract class methods
7. Partial abstraction
8. Interface-style design in Python
9. Python protocols vs Java interfaces

Hands-on: design a notification framework.

Phase 7 — Python object model and magic methods

This is where Python OOP becomes serious.

1. `__new__`
2. `__init__`
3. `__del__`
4. `__str__`
5. `__repr__`
6. `__eq__`
7. `__hash__`
8. `__lt__`
9. `__le__`
10. `__gt__`
11. `__len__`
12. `__bool__`
13. `__call__`
14. `__getitem__`
15. `__setitem__`
16. `__contains__`
17. `__iter__`
18. `__next__`

Hands-on: create our own container class.

Phase 8 — Attribute lookup internals

One of the most important advanced sections.

1. `object.__getattr__`
2. `__getattr__`
3. `__setattr__`
4. `__delattr__`

5. Attribute lookup algorithm
6. Instance dictionary
7. Class dictionary
8. MRO lookup
9. Descriptor priority
10. Infinite recursion problems

Hands-on: build an attribute logger and lazy-loading object.

Phase 9 — Descriptors

This is the machinery behind property, methods, and many frameworks.

1. Descriptor protocol
2. `__get__`
3. `__set__`
4. `__delete__`
5. Data descriptors
6. Non-data descriptors
7. Descriptor lookup priority
8. How functions become bound methods
9. How property works internally
10. Building our own property implementation

Hands-on: implement `MyProperty`.

This will also help you understand frameworks like Django and SQLAlchemy much better.

Phase 10 — Object creation and memory

1. `__new__` vs `__init__`
2. Object allocation
3. Object initialization
4. `id()`
5. Identity

6. Reference semantics
7. Mutable objects
8. Immutable objects
9. Reference counting
10. Garbage collection
11. Cyclic references
12. gc
13. Weak references
14. weakref

Java JVM comparison throughout.

Phase 11 — Metaclasses

The advanced part.

1. Classes are objects
2. type
3. type(obj)
4. type(Class)
5. Creating classes dynamically
6. type(name, bases, namespace)
7. Metaclass concept
8. type as the default metaclass
9. __prepare__
10. __new__ in metaclasses
11. __init__ in metaclasses
12. Class creation lifecycle
13. Metaclass conflict
14. Framework use cases

Hands-on: create a validation metaclass.

Phase 12 — Modern Python OOP

1. Dataclasses
2. @dataclass
3. field
4. default_factory
5. Frozen dataclasses
6. Ordering
7. Slots
8. __slots__
9. Memory optimization
10. Named tuples
11. Protocols
12. typing.Protocol
13. Structural subtyping
14. Generic classes
15. TypeVar
16. Generic inheritance

Hands-on: domain model design.

Phase 13 — Design principles

1. Composition vs inheritance
2. SOLID in Python
3. Dependency injection
4. Strategy pattern
5. Factory pattern
6. Singleton
7. Observer
8. Adapter
9. Decorator pattern
10. Repository pattern

Hands-on: build a mini backend architecture.